# Dynamic Collaboration Mode Selection in Hierarchical Federated Learning

Tingting Yang[a,b,c], Wenbo Zhang[a,b,c,*], Shaohua Liu[a,b,c], Lihua Zhang[a,b,c], Xinyuan Lin[a,b,c], Qian Liu[a,b,c]

[a] *School of Mathematical Sciences, Beijing University of Posts and Telecommunications, Beijing, China*
[b] *State Key Laboratory of Information Photonics and Optical Communications (Beijing University of Posts and Telecommunications), Beijing, China*
[c] *Key Laboratory of Mathematics and Information Networks (Beijing University of Posts and Telecommunications), Ministry of Education, Beijing, China*

**Abstract**

Hierarchical federated learning enables collaboration among Cloud, Edge, and end devices, but fixed collaboration schemes may not adapt effectively to heterogeneous and changing system conditions. To address this challenge, a dynamic collaboration mode selection framework is proposed. Three collaboration modes are considered: Edge aggregation with asynchronous Cloud updates ($FL_E$), multistage Edge aggregation followed by Cloud aggregation ($FL_{EC}$), and direct Cloud aggregation ($FL_C$). To evaluate candidate mode distributions, a historical aggregate update deviation score is constructed by combining mode specific historical update deviations with the aggregation weights of the corresponding Cloud update units. The proposed framework combines this score with estimated system latency into a joint utility and dynamically selects feasible mode distributions under resource and lifetime privacy constraints. Experiments on Fashion MNIST with LeNet 5 evaluate the relationship between the proposed deviation score and subsequent learning performance. Experimental results show that the rankings of the examined candidate distributions by the proposed score are consistent with their subsequent loss reductions, providing empirical support for the effectiveness of the proposed score. Further experiments demonstrate how different utility weights affect learning performance and cumulative system latency.

*Keywords:* hierarchical federated learning, cloud-edge-end collaboration, dynamic collaboration mode selection, differential privacy, device heterogeneity

## 1. Introduction

Federated learning (FL) enables multiple end devices to collaboratively train a shared model without transferring raw data to a central server [1, 2]. In AIoT systems, however, participating devices differ substantially in computational capability, memory capacity, communication quality, and local data distribution. Cloud-edge-end hierarchical federated learning (HFL) introduces edge servers between end devices and the cloud, providing intermediate aggregation and computation capabilities while reducing direct communication with the cloud [3, 4, 5].

Data and system heterogeneity can limit the effectiveness of fixed cloud-edge-end collaboration schemes across clients and training rounds. First, non-IID data can cause local updates to drift in different directions and can degrade simple weighted aggregation [6, 7, 8, 9, 10]. Second, system capabilities are heterogeneous. A weak end device may be unable to handle the same workload or meet the same deadline as a stronger device, whereas an edge or cloud server can undertake more computation [11, 12]. A mode that suits one client in one round may be inefficient or infeasible for the same client in another round, or for another client.

Existing work on HFL and cloud-edge-end collaboration has covered personalization, resource allocation, wireless scheduling, heterogeneous models, and split training [13, 14, 15, 16, 17, 18, 19, 20]. These studies address different aspects of collaborative training. However, heterogeneous client capabilities, non IID data, and changing communication conditions can affect the suitability of different collaboration modes across training rounds. A fixed mode assignment may therefore fail to maintain a desirable balance between learning related update behavior and system latency under resource and lifetime privacy constraints.

To address this problem, three collaboration modes are considered in the proposed framework. In $FL_E$, clients perform one local training stage and submit their updates to the Edge for aggregation. The resulting Edge model is returned to clients without waiting for Cloud aggregation, while the aggregated Edge update is asynchronously forwarded to the Cloud to contribute to global aggregation. In $FL_{EC}$, clients perform multiple local training and Edge aggregation stages before the final Edge update is forwarded to the Cloud. In $FL_C$, client updates are directly transmitted to the Cloud for aggregation.

These modes provide different aggregation paths and training stage arrangements, making their suitability dependent on the current system conditions. A mode distribution that is suitable under one set of conditions may not remain effective as client resources, communication conditions, and update characteristics change. Therefore, the proposed method dynamically selects a feasible distribution of clients among the three modes from a finite candidate set.

Each candidate is evaluated using a historical aggregate up-

*Corresponding author.
*Email address:* zhangwb@bupt.edu.cn (Wenbo Zhang)

date deviation score and estimated system latency, subject to resource and lifetime privacy constraints. These two quantities form the joint utility for mode selection. The deviation score serves as a decision signal rather than a direct measure of model accuracy. The learning performance of the selected strategies is then evaluated experimentally.

The main contributions of this paper are as follows:

- A dynamic collaboration mode selection framework is developed for hierarchical federated learning with three collaboration modes, $FL_E$, $FL_{EC}$, and $FL_C$, distinguished by their aggregation paths and training stage arrangements. A finite set of candidate mode distributions is used to represent different client assignments across these modes.
- A historical aggregate update deviation score is constructed by combining mode specific historical update deviations with the aggregation weights of the corresponding Cloud update units. The proposed score is integrated with estimated system latency into a normalized joint utility to evaluate feasible candidate mode distributions under resource and lifetime privacy constraints.
- Simulation experiments are conducted to evaluate the proposed framework. The results provide empirical support for the effectiveness of the proposed deviation score by showing consistency between candidate score rankings and subsequent loss reductions. Further comparisons characterize the tradeoffs between learning performance and system latency under different utility preferences and switching controls.

The remainder of this paper is organized as follows. Section 2 reviews related studies on hierarchical federated learning and cloud-edge-end collaboration. Section 3 introduces the system model and formulates the historical aggregate deviation contribution and system latency under resource and lifetime privacy constraints. Section 4 presents the proposed dynamic collaboration selection algorithm. Section 5 evaluates its performance through experiments. Finally, Section 6 concludes the paper.

## 2. Related Work

Existing research on hierarchical federated learning covers resource optimization, collaborative training and aggregation, and privacy protection. This section reviews representative studies in these three research directions.

### *2.1. HFL Under Heterogeneous Environments*

Hierarchical federated learning introduces intermediate Edge aggregation to improve communication efficiency in distributed training [3]. Resource constraints and differences in client capabilities have motivated the adaptation of training and communication parameters. PHFL employs model pruning and jointly adjusts pruning ratios, processor frequencies, and transmission power to address limited bandwidth and heterogeneous device capabilities [21]. Xu et al. investigate adaptive control of local update frequency and model compression to improve training efficiency [22].

More recently, Su et al. jointly optimize aggregation frequencies and computation and communication resources in hierarchical federated learning [23]. Wei et al. introduce a digital twin based framework that dynamically adjusts aggregation deadlines and resource allocation in hierarchical federated learning for IoT environments [24].

Data heterogeneity has also motivated client clustering and model personalization. Clustered FL organizes clients into groups according to similarities in their training characteristics [25]. Other studies investigate shared representations and personalized hierarchical federated learning to accommodate differences in local data distributions [26, 13]. Together with the resource management approaches discussed above, these studies address heterogeneous federated training through model adaptation, aggregation control, and resource optimization.

### *2.2. Cloud-Edge-End Collaborative Federated Learning*

Collaborative FL across the Cloud, Edge, and Local tiers can be organized through different training and aggregation mechanisms. FedAgg employs bridge samples and online knowledge distillation to transfer knowledge between models of different sizes across the three tiers [16]. FLaTEC distributes model components between devices and Edge servers and applies knowledge distillation to support efficient collaborative training [4]. Fed3Scale introduces a collaborative framework that combines training and aggregation across the three tiers with semi supervised learning [27].

Aggregation synchronization has also been investigated to improve training efficiency. FedSA employs a semi asynchronous mechanism that aggregates a specified number of received client updates instead of waiting for all participating clients [28]. This approach adjusts aggregation timing to reduce the waiting delay associated with synchronous training.

These studies illustrate different approaches to organizing collaborative training and aggregation across multiple tiers. The diversity of aggregation paths and synchronization requirements provides a basis for dynamic collaboration mode selection in hierarchical federated learning.

### *2.3. Privacy Preserving HFL*

Although raw training data remain on local devices, model updates may still reveal sensitive information about the training data [29]. Differential privacy (DP) has been widely investigated to limit information leakage during federated training [30].

Privacy preservation in HFL has been studied through different mechanisms. Chen et al. investigate personalized privacy protection using group local DP, incorporating randomized response and shuffling [31]. Xu et al. propose ADPFL, which adapts privacy budget allocation, gradient clipping, and client selection during training [32]. Wang et al. develop a client sampling framework that protects both communicated model

information and information revealed by client sampling decisions [33].

Other studies combine multiple privacy mechanisms within hierarchical architectures. Kalafatelis et al. integrate client level DP, secure aggregation, and homomorphic encryption for hierarchical learning across data spaces [34]. Islam et al. incorporate DP and personalization into hierarchical fog federated learning for IoT intrusion detection [35].

These studies investigate privacy preservation through different mechanisms and adaptive privacy control strategies. In the proposed framework, DP-SGD is applied during local training, and RDP accounting tracks the accumulated privacy loss across training rounds [36, 37]. The lifetime privacy budget is incorporated into the feasibility assessment of candidate collaboration mode distributions, together with resource constraints. Privacy constraints are therefore considered alongside historical update deviation and estimated system latency during mode selection.

## 3. Model Construction

### *3.1. Collaborative Mode and Transmission Modeling*

We consider a three-tier federated learning system consisting of $N$ clients, a set of edge servers, and a central cloud server. Each client $i$ is associated with an edge server $e(i)$. At round $t$, the system assigns a collaboration mode $h_i^t \in \mathcal{H}$ to each participating client, allowing training and aggregation to be organized across different tiers.

**Collaboration modes.** Three collaboration modes are considered in the cloud-edge-end architecture, which are distinguished by their aggregation paths and client-side synchronization requirements.

In Edge based Federated Learning ($FL_E$), clients perform local training and submit their updates to the Edge for aggregation. After Edge aggregation, the resulting Edge model is returned to clients independently of the Cloud aggregation process. Meanwhile, the aggregated Edge update is asynchronously forwarded to the Cloud and contributes to global aggregation. In Edge–Cloud Federated Learning ($FL_{EC}$), a group of clients associated with the same Edge participates in a multi-stage collaborative process. The Edge performs $E_B$ processing stages before submitting a single final joint update to the Cloud for global aggregation. In Cloud-based Federated Learning ($FL_C$), clients directly submit their locally trained updates to the Cloud for aggregation without intermediate Edge aggregation.

Let $L$, $E$, and $C$ denote the Local, Edge, and Cloud tiers, respectively. The three candidate collaboration modes are defined as

$$\mathcal{H} = \{\mathrm{FL_E}, \mathrm{FL_{EC}}, \mathrm{FL_C}\}. \tag{1}$$

In all three modes, raw training data remain on the client. Local model updates or Edge-aggregated updates are transmitted according to the selected collaboration mode.

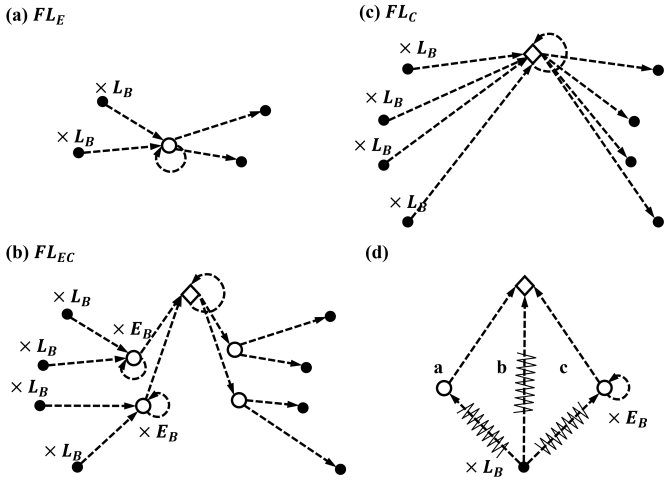



Figure 1: Execution structures of the three collaboration modes in the cloud-edge-end architecture.

#### *3.1.1. Collaborative Training Blocks and Mode Flows*

Let $L_B$ denote the number of local training steps in each block, and let $T_L(L_B)$ represent the corresponding local training process followed by model update generation. During local training, gradients computed for each example are clipped, and Gaussian noise is added to the clipped gradients using differentially private stochastic gradient descent. Let $A_E$ and $A_C$ denote aggregation at the Edge and Cloud, respectively. For the $FL_{EC}$ mode, $E_B$ denotes the number of Edge processing and aggregation stages. At each stage, clients perform $L_B$ local training steps using the latest model received from the Edge. After each stage, the aggregated model is returned to the clients for the next stage. The final aggregated update is denoted by $\Delta_e^{(E_B)}$.

$$\begin{aligned}
\mathrm{FL_E}&: \; T_L(L_B) \xrightarrow{\text{upd}} A_E \xrightarrow{\text{async upd}} A_C,\\
\mathrm{FL_{EC}}&: \; T_L(L_B) \xrightarrow{\text{upd}} \Delta_e^{(E_B)} \xrightarrow{\text{upd}} A_C,\\
\mathrm{FL_C}&: \; T_L(L_B) \xrightarrow{\text{upd}} A_C.
\end{aligned} \tag{2}$$

Here, The arrows indicate the direction of information flow. The label upd indicates the transmission of a model update to the next aggregation stage. The label async upd indicates asynchronous update transmission without requiring clients to wait for Cloud-side aggregation.

Figure 1 illustrates the execution structures of the three collaboration modes. Figures (a)–(c) show the execution and aggregation processes of $FL_E$, $FL_{EC}$, and $FL_C$, respectively, while Fig. (d) compares the three modes from the perspective of an individual client. Filled circles, hollow circles, and hollow diamonds represent the Local, Edge, and Cloud tiers, respectively. Dashed arrows indicate model update transmissions, and circular arrows represent aggregation operations. Spring-like marks indicate that the transmitted model updates contain noise introduced by client-side DP-SGD during local training.

### *3.2. Lifetime Privacy Accounting*

Sample-level differential privacy [38] is enforced during local training through the differentially private stochastic gradi-

ent descent mechanism [36], in which gradients are clipped and Gaussian noise is added to the clipped gradients. The resulting model updates carry this noise when transmitted across tiers. For each client, Rényi differential privacy (RDP) [37] is used to accumulate privacy loss across training rounds. Let $\mathcal{A}$ denote the set of Rényi orders [39]. For a Gaussian mechanism with noise multiplier $\sigma$, the RDP cost at order $\alpha \in \mathcal{A}$ is $\rho(\alpha, \sigma) = \alpha/(2\sigma^2)$.

Let $m_i^t$ denote the number of Gaussian mechanisms applied to client $i$ in round $t$, with corresponding noise multipliers $\sigma_{i,k}^t$. The accumulated RDP cost can be calculated as

$$\Phi_i^{t+1}(\alpha) = \Phi_i^t(\alpha) + \sum_{k=1}^{m_i^t} \rho(\alpha, \sigma_{i,k}^t). \tag{3}$$

For a given privacy parameter $\delta$, the corresponding privacy loss is

$$\epsilon_i^t = \min_{\alpha \in \mathcal{A}} \left[ \Phi_i^t(\alpha) + \frac{\log(1/\delta)}{\alpha - 1} \right]. \tag{4}$$

Let $\bar{\epsilon}$ denote the lifetime privacy budget of each client. The accumulated privacy loss must satisfy $\epsilon_i^t \le \bar{\epsilon}$ throughout training process.

For client $i$ and candidate mode $h$, let $N_i^t(h)$ denote the estimated number of Gaussian mechanisms whose privacy costs are attributed to client $i$ over the remaining training rounds. This estimate is determined by the planned local training schedule and the protected releases required by the selected mode. Let $R$ denote the total number of training rounds.

For $N_i^t(h) > 0$, the noise multiplier $\sigma_i^t(h)$ is calibrated to satisfy

$$\min_{\alpha \in \mathcal{A}} \left[ \Phi_i^t(\alpha) + \frac{N_i^t(h)\alpha}{2[\sigma_i^t(h)]^2} + \frac{\log(1/\delta)}{\alpha - 1} \right] \le \bar{\epsilon}. \tag{5}$$

A noise multiplier satisfying (5) is selected. If calibration is infeasible, the candidate mode is excluded. No calibration is required when $N_i^t(h) = 0$.

Before execution, the planned privacy cost is checked against the remaining budget. The privacy ledger is updated using the executed Gaussian mechanisms, and the resulting RDP state is used for mode selection in the next round.

### *3.3. Joint Cost and Utility Modeling*

Candidate mode distributions are evaluated using historical update deviation contributions and system latency, subject to resource constraints and the lifetime privacy budget in Sec. 3.2.

#### *3.3.1. Path and Round Latency on a Common Timeline*

Figure 2 provides two complementary views of collaboration latency. Fig. 2(a) illustrates the client-side execution and communication sequence of $FL_{EC}$, including the $E_B$ sequential Edge aggregation stages and the intermediate return transmissions from the Edge to the clients. After the final Edge aggregation stage, the resulting update is forwarded to the Cloud. Fig. 2(b) illustrates the system-level synchronization process for $FL_{EC}$ and $FL_C$. It shows how the final Edge updates and direct client updates determine the Cloud aggregation completion time, followed by the required return transmissions. For clarity, panel (b) uses $E_B = 1$ as an illustrative example, while the latency formulation supports general $E_B$. The asynchronous Cloud upload in $FL_E$ is omitted from panel (b) because it does not impose a waiting dependency on the Cloud-synchronized round.

**Transmission latency.** For a payload $b$ transmitted from tier $u$ to tier $v$, let $S_{u,v,b}^t(h_i)$ denote the original payload size under collaboration mode $h_i$. Let $B_{i,u,v}^t$ denote the transmission rate of the link between tiers $u$ and $v$ for client $i$ in round $t$, and let $D_{u,v}^0$ denote the fixed transmission delay of this link.

$$T_{u,v,b}^{\text{link},t}(h_i) = \frac{S_{u,v,b}^t(h_i)}{B_{i,u,v}^t} + D_{u,v}^0. \tag{6}$$

For an aggregated update transmitted from tier $u$ to tier $v$, let $S_{u,v,b}^t$ denote the payload size and $B_{u,v}^t$ denote the corresponding transmission rate. The transmission latency is

$$T_{u,v,b}^{\text{link},t} = \frac{S_{u,v,b}^t}{B_{u,v}^t} + D_{u,v}^0. \tag{7}$$

**Execution latency.** Let $\tau_{i,L}^t(h)$ denote the estimated execution time per local training step for client $i$ under mode $h$. The execution latency is given by

$$T_i^{\text{stage},t}(h) = L_B \tau_{i,L}^t(h), \quad h \in \{FL_E, FL_{EC}, FL_C\}. \tag{8}$$

Transmission and aggregation delays are accounted for separately.

**Reconfiguration latency.** Switching collaboration modes may require reconfiguring the communication and aggregation paths. We model the resulting reconfiguration delay as

$$T_i^{\text{sw},t} = \eta_h \mathbb{I}(h_i^t \ne h_i^{t-1}), \tag{9}$$

where $\eta_h$ denotes the mode switching overhead and $\mathbb{I}(\cdot)$ is the indicator function.

**Path arrival and aggregation.** For each candidate mode $h_i$, the mode flow determines the sequence of execution stages and payload transmissions leading to the relevant aggregation point. We calculate update arrival times by following this sequence and accounting for execution, transmission, and reconfiguration delays in their order of occurrence. Let $a_{i,E}^{t,k}(h)$ denote the arrival time of client $i$'s update at Edge stage $k$ under collaboration mode $h$. Let $a_{i,C}^t$ denote the arrival time of client $i$'s update at the Cloud under $FL_C$.

Given a mode distribution $\mathbf{h}^t = (h_1^t, \ldots, h_N^t)$, clients are grouped by their associated Edge and selected mode. Let

$$\mathcal{G}_{g,m}^t = \{i \mid e(i) = g,\ h_i^t = m\}, \quad m \in \{FL_E, FL_{EC}\}, \tag{10}$$

denote the clients assigned to mode $m$ at Edge $g$. Let $k$ denote the Edge aggregation stage index. For $FL_E$, only one Edge aggregation stage is performed, whereas $FL_{EC}$ consists of $E_B$ sequential Edge aggregation stages.

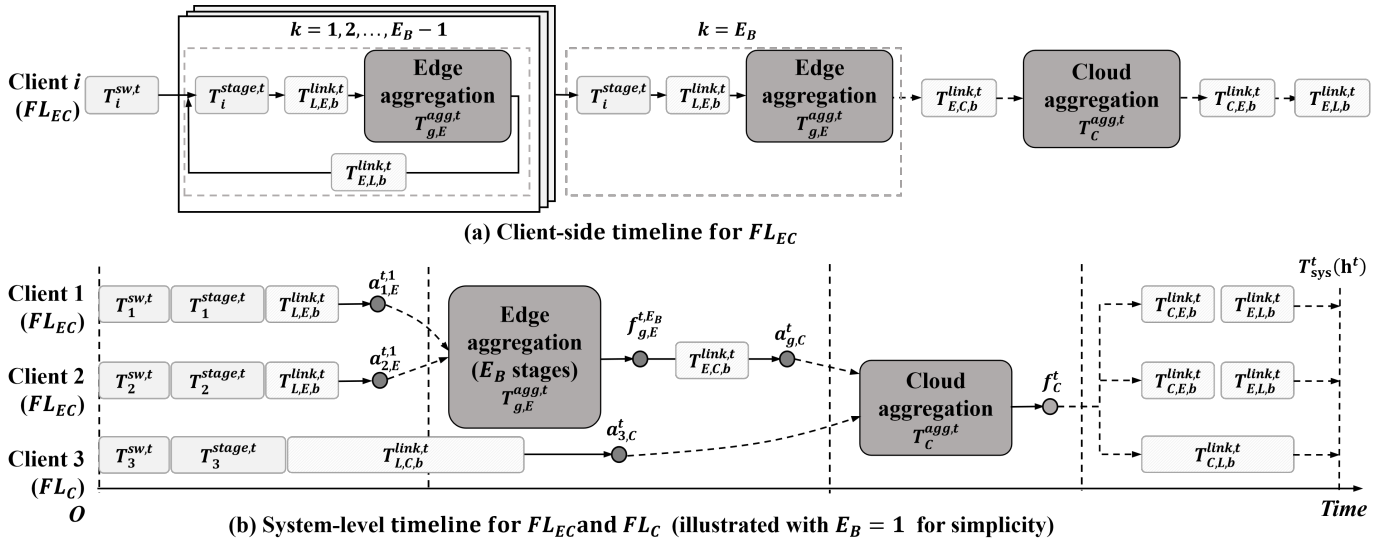



Figure 2: Client-side and system-level latency timelines. (a) Client-side execution and communication path for $FL_{EC}$, showing the $E_B$ sequential Edge aggregation stages. (b) System-level synchronization timeline for $FL_{EC}$ and $FL_C$, illustrated with $E_B = 1$ for clarity. Cloud aggregation waits for the final $FL_{EC}$ Edge updates and direct $FL_C$ client updates, followed by the required return transmissions. The asynchronous Cloud upload in $FL_E$ is omitted because it does not delay the Cloud-synchronized round.

For $h \in \{FL_E, FL_{EC}\}$, the update arrival time at the first Edge stage is

$$a_{i,E}^{t,1}(h) = T_i^{\mathrm{sw},t} + T_i^{\mathrm{stage},t}(h) + T_{L,E,b}^{\mathrm{link},t}(h). \tag{11}$$

For a nonempty Edge group $\mathcal{G}_{g,m}^t$, the completion time of aggregation stage $k$ is

$$f_{g,E}^{t,k}(m) = \max_{i \in \mathcal{G}_{g,m}^t} a_{i,E}^{t,k}(m) + T_{g,E}^{\mathrm{agg},t}, \tag{12}$$

where $k = 1$ for $FL_E$ and $k = 1, \ldots, E_B$ for $FL_{EC}$. Updates arriving earlier remain buffered until all required updates in the same aggregation group have arrived.

For $FL_{EC}$, each subsequent Edge stage starts after the aggregated model from the previous stage has been returned to the clients. The clients then perform another local training block and transmit their updates to the Edge. Thus, for $k = 1, \ldots, E_B - 1$, the update arrival time at the next stage is

$$\begin{aligned} a_{i,E}^{t,k+1}(FL_{EC}) = f_{g,E}^{t,k}(FL_{EC}) + T_{E,L,b}^{\mathrm{link},t}(FL_{EC}) \\ + T_i^{\mathrm{stage},t}(FL_{EC}) + T_{L,E,b}^{\mathrm{link},t}(FL_{EC}). \end{aligned} \tag{13}$$

For $FL_E$, the aggregated Edge update is forwarded to the Cloud asynchronously after the single Edge aggregation stage. This transmission does not require clients to wait for Cloud aggregation and is excluded from the Cloud synchronization latency.

For $FL_{EC}$, the final Edge update is forwarded to the Cloud after the $E_B$-th Edge aggregation stage. Its arrival time is

$$a_{g,C}^t = f_{g,E}^{t,E_B}(FL_{EC}) + T_{E,C,b}^{\mathrm{link},t}. \tag{14}$$

For $FL_C$, clients transmit their updates directly to the Cloud. The arrival time of client $i$'s update is

$$a_{i,C}^t = T_i^{\mathrm{sw},t} + T_i^{\mathrm{stage},t}(FL_C) + T_{L,C,b}^{\mathrm{link},t}(FL_C). \tag{15}$$

Cloud aggregation waits for the direct client updates from $FL_C$ and the final Edge updates from $FL_{EC}$. Let $\mathcal{G}_{EC}^t = \{g \mid \mathcal{G}_{g,FL_{EC}}^t \neq \emptyset\}$ denote the Edge servers with active $FL_{EC}$ groups. The Cloud aggregation completion time is

$$\begin{aligned} f_C^t = \max \left\{ \max_{i \in \mathcal{D}_C^t} a_{i,C}^t, \max_{g \in \mathcal{G}_{EC}^t} a_{g,C}^t \right\} \\ + T_C^{\mathrm{agg},t}, \end{aligned} \tag{16}$$

where $\mathcal{D}_C^t$ denotes the set of $FL_C$ clients. The asynchronous uploads of $FL_E$ do not affect the Cloud synchronization time.

The system round latency is measured with respect to Cloud synchronization. It is determined by the completion times of $FL_{EC}$ and $FL_C$ clients, including the required return transmissions. The asynchronous Cloud uploads of $FL_E$ do not impose a waiting dependency on the Cloud synchronization process. The system round latency is therefore

$$\begin{aligned} T_{\mathrm{sys}}(\mathbf{h}^t) = f_C^t + \max \left\{ \max_{i \in \mathcal{D}_{EC}^t} \left( T_{C,E,b}^{\mathrm{link},t}(h_i^t) + T_{E,L,b}^{\mathrm{link},t}(h_i^t) \right), \right. \\ \left. \max_{i \in \mathcal{D}_C^t} T_{C,L,b}^{\mathrm{link},t}(h_i^t) \right\}. \end{aligned} \tag{17}$$

Here, $\mathcal{D}_{EC}^t = \{i \mid h_i^t = FL_{EC}\}$ and $\mathcal{D}_C^t = \{i \mid h_i^t = FL_C\}$ denote the corresponding client sets. Clients operating in $FL_E$ are excluded from both sets because they do not wait for Cloud aggregation. Return delays are evaluated using (6), with absent return paths excluded from the maximum.

The client-side latency for each collaboration mode is defined as

$$T_{i,\mathrm{client}}^t(h_i^t) = \begin{cases} f_{g,E}^{t,1}(FL_E) + T_{E,L,b}^{\mathrm{link},t}(h_i^t), & h_i^t = FL_E, \\ f_C^t + T_{C,E,b}^{\mathrm{link},t}(h_i^t) + T_{E,L,b}^{\mathrm{link},t}(h_i^t), & h_i^t = FL_{EC}, \\ f_C^t + T_{C,L,b}^{\mathrm{link},t}(h_i^t), & h_i^t = FL_C. \end{cases} \tag{18}$$

*3.3.2. Historical Global Update Deviation Modeling*

Different collaboration modes involve distinct training and aggregation procedures, which may result in different update characteristics. The historical update deviations of each mode are estimated from previous rounds and used to evaluate the aggregate deviation signal associated with each candidate mode distribution.

**Cloud aggregation and weights.** For a candidate distribution $\mathbf{h}^t = (h_1^t, \ldots, h_N^t)$, clients connected to the same Edge and assigned the same mode are grouped. Let $\mathcal{K}^t(\mathbf{h}^t)$ denote the units included in the current Cloud aggregation. Each $FL_{EC}$ group contributes one final Edge update after $E_B$ stages, while each $FL_C$ client contributes its own update directly. For $FL_E$, each Edge group sends its aggregated update to the Cloud asynchronously. The update is included in the Cloud aggregation that accepts it, without delaying the current synchronization.

Figure 3 illustrates how mode reassignment changes the grouping structure and the resulting Cloud update units. From Fig. 3(a) to Fig. 3(b), client $l_2$ switches from $FL_{EC}$ to $FL_C$, while client $l_5$ switches from $FL_E$ to $FL_{EC}$.

For example, if the two configurations are applied in consecutive rounds $t-1$ and $t$, respectively, the set of Cloud update units changes from

$$\mathcal{K}^{t-1}(\mathbf{h}^{t-1}) = \{k_1, k_2, k_3, k_4, k_5, k_6\} \tag{19}$$

to

$$\mathcal{K}^{t}(\mathbf{h}^{t}) = \{k_1, k_2, k_3, k_4, k_5, k_6, k_7\}. \tag{20}$$

Here, $k_i$ denotes the $i$th Cloud update unit in the corresponding round. The units are indexed separately in each round, and their composition depends on the client mode distribution.

Let $n_i$ denote the number of training samples of client $i$, and let $\mathcal{M}_k^t$ denote the clients represented by Cloud unit $k$. The aggregation weight is calculated as

$$\begin{aligned} N_k^t &= \sum_{i \in \mathcal{M}_k^t} n_i, \\ I_k^t(\mathbf{h}^t) &= \frac{N_k^t}{\sum_{\ell \in \mathcal{K}^t(\mathbf{h}^t)} N_\ell^t}. \end{aligned} \tag{21}$$

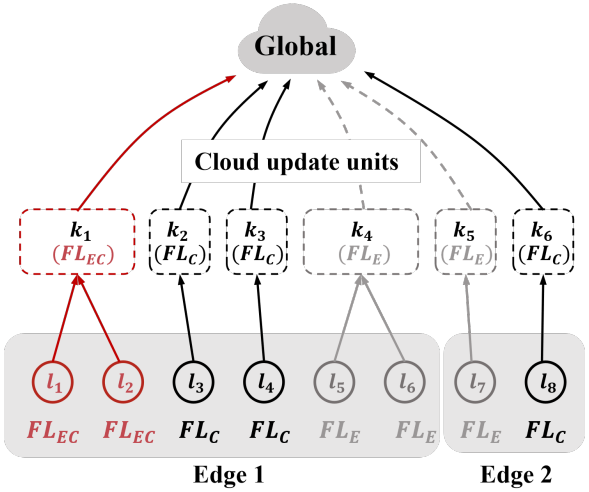
For example, in Fig. 3(a), unit $k_1$ represents clients $l_1$ and $l_2$, with $N_{k_1}^{t-1} = n_1 + n_2$. In Fig. 3(b), unit $k_1$ instead represents clients $l_1$ and $l_5$, with $N_{k_1}^{t} = n_1 + n_5$. Meanwhile, client $l_2$ forms an individual Cloud update unit $k_2$, with $N_{k_2}^{t} = n_2$. The aggregation weights are determined by these sample counts and the total samples represented by the accepted Cloud units.

**Historical deviation by mode.** Let $\Delta_C^\tau$ denote the Cloud update at round $\tau$. The historical global reference is calculated from previous Cloud updates as

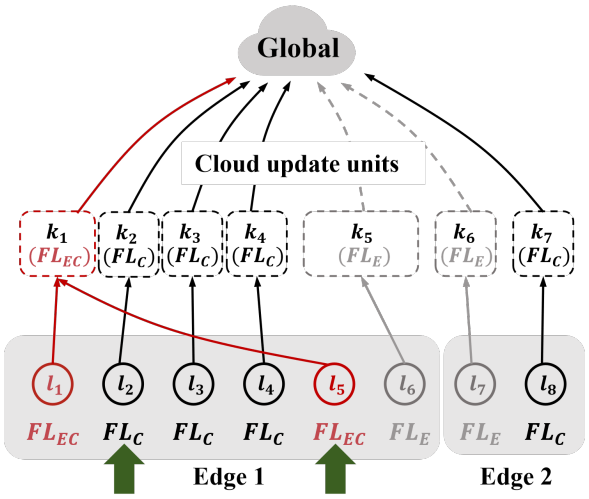
$$v_g^{\tau-1} = \gamma_C v_g^{\tau-2} + (1-\gamma_C)\Delta_C^{\tau-1}, \quad 0 \le \gamma_C < 1. \tag{22}$$

The first available Cloud update is used to initialize the reference.

Let $u_{k,\mathrm{DP}}^\tau$ denote the update of unit $k$ accepted by the Cloud at round $\tau$ after the required privacy protection. Only updates




(a) Before mode reassignment




(b) After mode reassignment

Figure 3: Illustration of client grouping and Cloud update units under different mode distributions. From (a) to (b), client $l_2$ switches from $FL_{EC}$ to $FL_C$, while client $l_5$ switches from $FL_E$ to $FL_{EC}$. These changes alter the grouping structure and the resulting Cloud update units.

allowed under Sec. 3.2 are used. The deviation of unit $k$ from the historical reference is

$$d_k^\tau = u_{k,\mathrm{DP}}^\tau - v_g^{\tau-1}. \tag{23}$$

Let $\mathcal{K}_m^\tau$ denote the Cloud units of mode $m \in \mathcal{H}$ included at round $\tau$. If this set is not empty, the weighted mean deviation of mode $m$ is

$$\bar{d}_m^\tau = \frac{\sum_{k \in \mathcal{K}_m^\tau} I_k^\tau d_k^\tau}{\sum_{k \in \mathcal{K}_m^\tau} I_k^\tau}. \tag{24}$$

The weighted mean is calculated for each mode across all Edges.

The historical mean deviation of each mode is estimated using an exponential moving average (EMA):

$$\widehat{d}_m^t = \gamma_d \widehat{d}_m^{t-1} + (1-\gamma_d)\bar{d}_m^{t-1}, \quad 0 \le \gamma_d < 1. \tag{25}$$

The first available mean deviation is used to initialize the estimate. If no new data are available, the previous estimate is

retained. As clients switch modes between rounds, the set of clients assigned to the same mode at each Edge may change. Therefore, the mean deviation is estimated for each mode rather than for a fixed group of clients.

**Update deviation score.** For each candidate mode distribution $\mathbf{h}^t$, the Cloud units and their aggregation weights are determined. The historical deviation contributions are aggregated and combined with the global reference as follows:

$$\widehat{U}_C^t(\mathbf{h}^t) = v_g^{t-1} + \sum_{k \in \mathcal{K}^t(\mathbf{h}^t)} I_k^t(\mathbf{h}^t) \widehat{d}_{m(k)}^t, \tag{26}$$

Here, $m(k)$ denotes the collaboration mode of Cloud update unit $k$. For example, in Fig. 3(b), $m(k_1) = FL_{EC}$ and $m(k_2) = FL_C$. Their contributions to the aggregate deviation are weighted by $I_{k_1}^t(\mathbf{h}^t)$ and $I_{k_2}^t(\mathbf{h}^t)$, respectively. The update deviation score measures the normalized squared magnitude of the aggregate historical deviation contribution:

$$J^t(\mathbf{h}^t) = \frac{\left\| \widehat{U}_C^t(\mathbf{h}^t) - v_g^{t-1} \right\|_2^2}{\left\| v_g^{t-1} \right\|_2^2 + \varepsilon_J}, \quad \varepsilon_J > 0. \tag{27}$$

A larger $J^t(\mathbf{h}^t)$ indicates a greater historical aggregate update deviation under the candidate mode distribution. Therefore, larger values of $J^t(\mathbf{h}^t)$ are preferred to prioritize mode distributions with more pronounced historical update characteristics. The system latency is also considered to balance the historical aggregate update deviation score and the execution overhead.

### *3.4. Finite Candidate Joint Selection*

Let $P_j$ denote a candidate mode distribution, where $j = 0, \ldots, M-1$ and $M$ is the number of candidate distributions considered. Each candidate specifies the proportions of clients assigned to $(FL_E, FL_{EC}, FL_C)$.

For each candidate $P_j$, let $\mathbf{h}_j^t = \Psi^t(P_j)$ denote the corresponding client mode assignment in round $t$, where $\Psi^t$ represents the assignment procedure applied at each Edge.

Let $\mathcal{F}_{\text{sys}}^t$ denote the set of client mode assignments satisfying the resource, privacy, and system admission constraints in round $t$. The feasible candidate set is defined as

$$C^t = \left\{ \mathbf{h}_j^t \mid 0 \le j < M,\ \mathbf{h}_j^t \in \mathcal{F}_{\text{sys}}^t \right\}. \tag{28}$$

The search complexity is $O(M)$, as at most $M$ candidate distributions are considered for joint utility maximization.

The candidate set can be extended by considering additional mode distributions. For experimental evaluation, five candidate mode distributions are considered, with $M = 5$. Their proportions for $(FL_E, FL_{EC}, FL_C)$ are specified as

$$\begin{aligned} &P_0 = (0.30, 0.30, 0.40), \quad P_1 = (0.50, 0.25, 0.25), \\ &P_2 = (0.25, 0.50, 0.25), \quad P_3 = (0.25, 0.25, 0.50), \\ &P_4 = (0.40, 0.40, 0.20). \end{aligned}$$

The specified proportions are applied separately to the clients associated with each Edge. For each candidate, the number of clients assigned to each mode is obtained by rounding the corresponding proportion and adjusting the resulting counts to match the total number of clients at that Edge.

For each feasible assignment $\mathbf{h}_j^t \in C^t$, the historical update deviation score $J^t(\mathbf{h}_j^t)$ and system latency $T_{\text{sys}}(\mathbf{h}_j^t)$ are evaluated. All candidates are evaluated using the historical statistics and system conditions available in round $t$.

Since the two quantities have different scales, their values are normalized among the feasible candidates in each round. Let $J_{\min}^t$ and $J_{\max}^t$ denote the minimum and maximum values of $J^t$. Similarly, $T_{\min}^t$ and $T_{\max}^t$ denote the minimum and maximum system latencies.

The joint utility is defined as

$$\begin{aligned} U^t(\mathbf{h}^t; \lambda) = \lambda \frac{J^t(\mathbf{h}^t) - J_{\min}^t}{J_{\max}^t - J_{\min}^t} \\ + (1-\lambda) \frac{T_{\max}^t - T_{\text{sys}}(\mathbf{h}^t)}{T_{\max}^t - T_{\min}^t}, \end{aligned} \tag{29}$$

where $0 \le \lambda \le 1$ controls the relative importance of the historical deviation contribution and system latency. A normalized value of 0.5 is used when the corresponding maximum and minimum are equal.

A larger $J^t(\mathbf{h}^t)$ is preferred to favor stronger historical aggregate deviation contributions. A smaller $T_{\text{sys}}(\mathbf{h}^t)$ is preferred to reduce system execution time. Both preferences are therefore represented by larger normalized values in the joint utility.

The mode distribution is selected by

$$\mathbf{h}^{t,*} \in \arg\max_{\mathbf{h}^t \in C^t} U^t(\mathbf{h}^t; \lambda). \tag{30}$$

If multiple candidates achieve the same maximum utility, the candidate with the smallest index is selected.

## 4. Algorithm

Algorithm 1 presents the dynamic collaboration mode selection procedure, and the main symbols are summarized in Table 1. At each mode selection step, at most $M$ candidate distributions are evaluated based on their feasibility, historical deviation contribution, and system latency. The candidate search complexity is $O(M)$ in terms of the number of candidate evaluations.

**Algorithm 1** Dynamic Collaboration Mode Selection

**Require:** Round $t$, candidate distributions $\{P_j\}_{j=0}^{M-1}$, strategy period $S_P$, utility weight $\lambda$, hysteresis threshold $\theta_H$, previous candidate index $j_{t-1}$, system state, privacy ledger, and historical statistics
**Ensure:** Selected mode distribution $\mathbf{h}^{t,*}$ or SKIP

```
if t ≤ 5 then
    h_0^t ← Ψ^t(P_0)
    if h_0^t ∉ F_sys^t then
        return SKIP
    end if
    return h_0^t
end if
if t > 1, mod(t − 1, S_P) ≠ 0, and h^{t−1,*} ∈ F_sys^t then
```

Table 1: Notation Used in Algorithm 1

| Symbol | Description |
|---|---|
| $M$ | Number of candidate mode distributions. |
| $P_j$ | Candidate mode distribution indexed by $j$. |
| $S_P$ | Mode selection period. |
| $\mathcal{F}_{\text{sys}}^t$ | Set of system feasible mode distributions. |
| $C^t$ | Set of feasible candidate distributions in round $t$. |
| $\mathbf{h}^{t,*}$ | Selected mode distribution in round $t$. |
| $J^t(\mathbf{h}^t)$ | Historical aggregate deviation contribution. |
| $T_{\text{sys}}(\mathbf{h}^t)$ | System latency under distribution $\mathbf{h}^t$. |
| $U^t(\mathbf{h}^t;\lambda)$ | Joint utility for candidate selection. |

```
    h^{t,*} ← h^{t−1,*}
else
    Construct C^t using (28)
    if C^t = ∅ then
        return SKIP
    end if
    Freeze v_g^{t−1} and {d̂_m^t}_{m∈H}
    for all h_j^t ∈ C^t do
        Evaluate J^t(h_j^t) using (27)
        Evaluate T_sys(h_j^t) using (17)
    end for
    Determine J^t_min, J^t_max, T^t_min, and T^t_max from all feasible candidates
    for all h_j^t ∈ C^t do
        Compute U^t(h_j^t; λ) using (29)
    end for
    Select h^{t,*} using (30)
    Resolve utility ties by selecting the candidate with the smallest index j
    if θ_H > 0 and t > 1 then
        if h^t_{j_{t−1}} ∈ C^t then
            if U^t(h^{t,*}; λ) − U^t(h^t_{j_{t−1}}; λ) ≤ θ_H then
                h^{t,*} ← h^t_{j_{t−1}}
            end if
        end if
    end if
end if
return h^{t,*}
```

## 5. Experiments

The experiments test whether the proposed dynamic collaboration mode selection method, which jointly considers the historical aggregate deviation contribution $J^t(\mathbf{h}^t)$ and system latency $T_{\text{sys}}(\mathbf{h}^t)$, achieves a more favorable tradeoff between test loss and cumulative system latency than fixed collaboration mode distributions. Five fixed strategies based on $P_0, \ldots, P_4$ are compared with seven dynamic strategies: three Greedy variants with $\lambda \in \{0.25, 0.50, 0.75\}$, Maximum $J$, Minimum $T$, Hysteresis, and Periodic selection. The comparisons cover convergence, communication and computation costs, and candidate selection, with particular attention to the effects of Hysteresis and Periodic selection on mode switching.

### 5.1. Experimental Setup

**Data and training.** Fashion MNIST contains 60,000 training images and 10,000 test images. To simulate heterogeneous data distributions, the training samples of each class are allocated to ten Edge servers using a Dirichlet distribution with concentration parameter $\alpha = 0.5$. Each Edge server then distributes its assigned samples among ten clients. A LeNet 5 model is trained for 100 global rounds. The training parameters and collaboration mode configurations are summarized in Table 2. The reverse links are assumed to have the same nominal transmission rates and fixed delays as the corresponding forward links.

**System conditions.** To model differences in client computing capabilities, each client is assigned a computation factor sampled from $\mathcal{U}(0.5, 2.0)$ and held constant across rounds. The local computation time per step is 0.1 s multiplied by this factor, while the client compute capacity is 2.0 divided by the same factor. Client memory is independently assigned by multiplying 32 MB by a fixed draw from $\mathcal{U}(0.5, 2.0)$. All 100 clients satisfy the resource requirements and participate in every training round. The same client population is used across strategies to compare different collaboration mode distributions. The system resource settings are listed in Table 2.

**System latency.** Different collaboration modes involve different computation, communication, and aggregation processes, which lead to different system latencies. To capture these differences, an event based simulation is used to model the execution timeline of each collaboration mode. All strategies are evaluated using the same system conditions and timing rules to provide a consistent comparison of system efficiency. Communication latency is calculated using Eq. (6), while local execution, Edge aggregation, Cloud aggregation, and mode reconfiguration follow Eqs. (8), (12), (16), and (9), respectively. The transmission rates, link delays, and other latency parameters are summarized in Table 2.

**Privacy configuration.** Local training applies per example gradient clipping with $C = 1.0$, followed by Gaussian noise addition at each local update step. Under replacement adjacency, the sensitivity is $2C$, and the noise standard deviation is $2C\sigma$. Privacy expenditure is tracked using an RDP accountant without sampling amplification. The privacy parameters are summarized in Table 2. The noise multiplier is calibrated according to the selected collaboration mode and the remaining lifetime privacy budget, as specified in (5). In the reported experimental configuration, the representative values are approximately 9.803 for $FL_E$ and $FL_C$, and 16.979 for $FL_{EC}$.

### 5.2. Comparison Strategies and Evaluation Metrics

**Main comparison.** All fixed and dynamic strategies use the five candidate distributions $P_0, \ldots, P_4$ defined in Sec. 3.4. Each fixed strategy retains one candidate throughout training, while dynamic strategies select candidates according to the joint utility or its individual objective components. The Greedy variants maximize $U^t(\mathbf{h}^t;\lambda)$ with $\lambda \in \{0.25, 0.50, 0.75\}$, while Maximum $J$ and Minimum $T$ prioritize the respective objective components. Hysteresis and Periodic selection use $\lambda = 0.50$ with

Table 2: Experimental configuration.

| Parameter | Configuration |
|---|---|
| *Data and training* | |
| Dataset | Fashion MNIST (60,000 training, 10,000 test) |
| Model | LeNet 5 |
| Topology | 100 clients, 10 Edge servers |
| Data heterogeneity | Dirichlet distribution, $\alpha = 0.5$ |
| Global rounds | 100 |
| Local optimization | DP SGD, batch size 64, learning rate 0.05 |
| Local steps | 2 per training stage |
| Mode stages | $FL_E$: 1, $FL_{EC}$: 3, $FL_C$: 1 |
| *System conditions* | |
| Local computation | 0.1 s per step, factor $\mathcal{U}(0.5, 2.0)$ |
| Client memory | 32 MB, factor $\mathcal{U}(0.5, 2.0)$ |
| Client participation | All 100 clients per round |
| *System latency* | |
| Link rates | $L \to E$: 5.0, $L \to C$: 2.2, $E \to C$: 8.0 MB/s |
| Rate variation | $\mathcal{U}(0.75, 1.25)$ per client and round |
| Link delays | $L \to E$: 0.015, $L \to C$: 0.04, $E \to C$: 0.01 s |
| Edge aggregation | 0.02 s per group and stage |
| Cloud aggregation | 0.04 s per aggregation |
| Mode switching | 0.02 s per changed client |
| *Privacy configuration* | |
| Clipping norm | $C = 1.0$ |
| Noise multiplier | Mode and budget dependent; $FL_E$, $FL_C$: $\sigma \approx 9.803$; $FL_{EC}$: $\sigma \approx 16.979$ |
| Privacy budget | $\epsilon \leq 8$, $\delta = 10^{-5}$ |

additional restrictions on reconfiguration. Hysteresis requires a utility improvement greater than 0.05 before switching candidates, whereas Periodic selection allows a new decision every five rounds. These strategies examine how restrictions on mode switching affect learning performance and system latency.

**Pure mode references.** Three pure mode strategies are considered, where all clients use the same collaboration mode throughout training: $FL_E$, $FL_{EC}$, or $FL_C$. These strategies are evaluated separately from the five candidate distributions to compare the learning performance and system costs of different collaboration modes.

All dynamic strategies first execute $P_0$ for five rounds. Its relatively balanced proportions allow historical update information to be collected from all three collaboration modes. Candidate selection then begins in round 6 using the accumulated historical statistics.

**Metrics.** Learning performance is evaluated using test loss and test accuracy. System efficiency is measured by cumulative system latency, total communication volume in MB, and the number of local training steps. Candidate changes and client mode changes are recorded to evaluate mode switching behavior.

### 5.3. Learning Performance and Efficiency

The five fixed candidate distributions are first compared by test loss and cumulative system latency. The values of the historical aggregate deviation contribution $J$ and system latency are then compared for feasible candidates at the first online decision. Finally, the dynamic selection strategies are evaluated over training. For the main experiments, results are reported as means across seeds 42, 43, and 44, with shaded bands or error bars indicating one sample standard deviation.

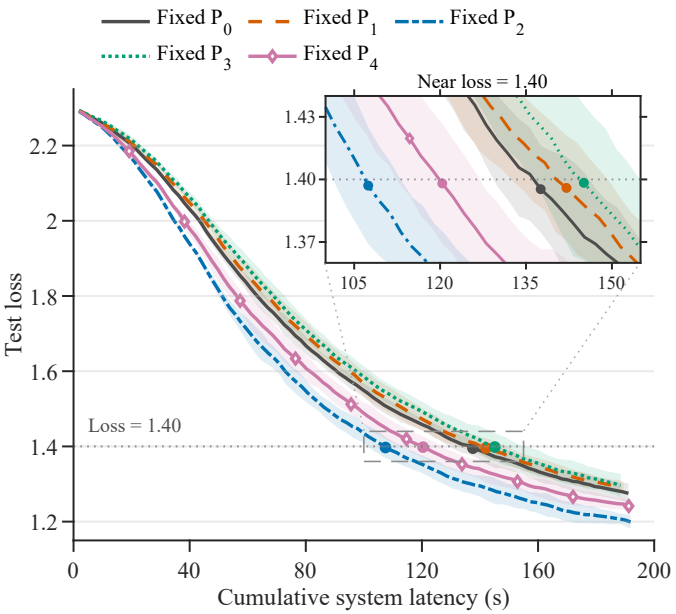



Figure 4: Test loss versus cumulative system latency for fixed strategies. Inset circles mark the first recorded mean loss at or below 1.40.

Table 3: Client mode proportions and cumulative latency at the circled points in Fig. 4. Latency is reported as mean ± sample standard deviation.

| Candidate | $FL_E$ | $FL_{EC}$ | $FL_C$ | Cumulative latency (s) |
|---|---|---|---|---|
| Fixed $P_0$ | 30% | 30% | 40% | 137.56 ± 1.55 |
| Fixed $P_1$ | 50% | 25% | 25% | 142.07 ± 1.13 |
| Fixed $P_2$ | 25% | 50% | 25% | 107.45 ± 0.87 |
| Fixed $P_3$ | 25% | 25% | 50% | 145.10 ± 4.30 |
| Fixed $P_4$ | 40% | 40% | 20% | 120.38 ± 1.54 |

#### 5.3.1. Fixed Candidate Learning Performance

Figure 4 compares Fixed $P_0$ to Fixed $P_4$ by test loss and cumulative system latency to assess their learning progress relative to the simulated time required. The mean final losses increase in the order $P_2$, $P_4$, $P_0$, $P_1$, and $P_3$. Fixed $P_2$ obtains the lowest mean final loss, 1.1992 ± 0.0188, and the highest mean accuracy over rounds 91–100 among the fixed strategies, 54.97 ± 0.78%.

The inset highlights the latency differences near loss 1.40. The mean curves first reach this level in the order $P_2$, $P_4$, $P_0$, $P_1$, and $P_3$, as summarized in Table 3. Fixed $P_2$ reaches this reference level at 107.45 s, compared with 137.56 s for Fixed $P_0$, a reduction of approximately 21.9%. These results show that $P_2$ and $P_4$, which assign 50% and 40% of clients to $FL_{EC}$, respectively, achieve lower mean final losses than the other fixed configurations. However, these configurations also execute more local training steps. Fixed $P_2$ and $P_4$ execute 400 and 360 steps per round, compared with 320 for $P_0$ and 300 for $P_1$ and $P_3$. Their results therefore reflect both the collaboration configuration and the amount of training performed.

#### 5.3.2. Candidate Scores and Training Tradeoffs

Figure 5 compares candidate scores with subsequent learning performance from a common training state. After five rounds of $P_0$, all five candidates are evaluated using the same

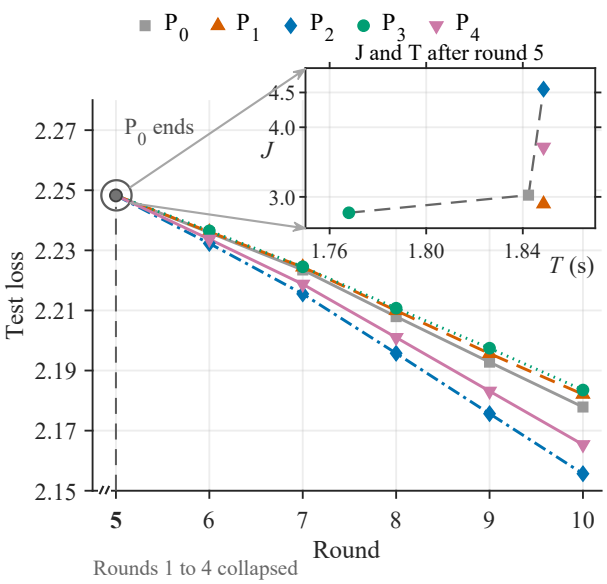



Figure 5: Candidate scores and subsequent test loss following five common rounds of $P_0$. The circled point marks the end of round 5. Each continuation uses one candidate in rounds 6 to 10. The inset shows historical deviation contribution $J$ and system latency $T$ in seconds, evaluated at the common round 5 state. Its dashed line connects the nondominated candidate scores. Results are obtained using seed 42.

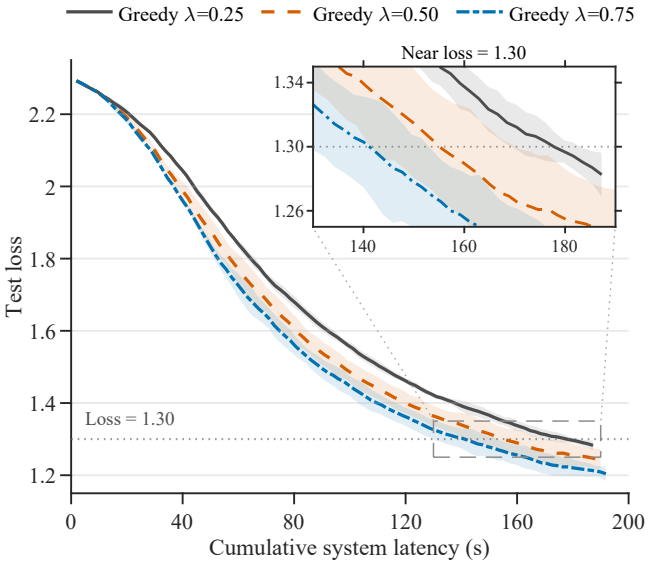



Figure 6: Test loss versus cumulative system latency for the three Greedy weights. Curves show mean loss against mean cumulative latency at matching rounds across three seeds. The inset enlarges the region near loss 1.30.

model and historical information. Each candidate is then applied separately in rounds 6 to 10. This preliminary experiment uses seed 42, with results obtained from a single run for each candidate. The inset shows the corresponding $J$ and system latency $T$, while the main plot shows the subsequent test loss.

The inset identifies $P_3$, $P_0$, and $P_2$ as nondominated candidates at the common decision state. Among them, $P_2$ provides the largest $J$, whereas $P_3$ has the smallest system latency $T$. $P_0$ has intermediate values of both objectives, offering a compromise between historical deviation contribution and system latency.

The main plot shows the subsequent learning outcomes when these candidates are retained for five rounds. From round 5 to round 10, the loss reductions for $P_2$, $P_0$, and $P_3$ are 0.0926, 0.0704, and 0.0648, respectively, following the same ordering as their initial $J$ values. Meanwhile, the recorded cumulative system latencies at round 10 are 18.8601, 18.8527, and 18.4668 s, respectively. Thus, $P_2$ achieves the largest loss reduction, while $P_3$ requires the least system time. This ordering is consistent with the fixed strategy results in Fig. 4, where $P_2$ achieves the lowest final loss among the fixed candidates. These observations provide preliminary empirical support for using $J$ as an indicator of potential learning progress when comparing candidate mode distributions.

### 5.4. Dynamic Selection and Switching Control

The preceding comparison motivates using $J$ together with system latency $T$ in the joint utility $U$ defined in (29). The values $\lambda = 0.25$, 0.50, and 0.75 represent a stronger preference for lower latency, equal weighting of the two normalized terms, and a stronger preference for historical deviation contribution, respectively. Starting from round 6, Greedy evaluates the feasible candidates in every round and selects the distribution with the highest current $U$, allowing candidate choices to adapt as training progresses. This subsection examines how $\lambda$ influences the learning and latency tradeoffs, and how hysteresis and periodic updates regulate mode switching.

#### 5.4.1. Effect of the Joint Utility Weight

Figure 6 examines how the preference between historical deviation contribution and system latency affects learning efficiency. The three Greedy strategies share the same five rounds of $P_0$ initialization within each seed and differ in the weight used for subsequent candidate selection.

The curves show faster mean loss reduction as $\lambda$ increases over the three tested values, with the inset highlighting their separation near loss 1.30. Greedy $\lambda = 0.25$, 0.50, and 0.75 first reach this threshold at mean rounds $94.7 \pm 3.8$, $83.0 \pm 7.8$, and $74.0 \pm 6.0$, with mean cumulative latencies $177.28 \pm 10.03$, $157.60 \pm 16.21$, and $141.95 \pm 11.82$ s, respectively. Thus, a stronger preference for lower candidate latency in each round does not necessarily reduce the cumulative time needed to reach a given loss. Among the tested weights, $\lambda = 0.75$ provides the earliest mean crossing of loss 1.30.

#### 5.4.2. Objective Preferences and Switching Control

Figure 7 compares Greedy $\lambda = 0.50$ with Maximum $J$ and Minimum $T$ to examine the learning consequences of emphasizing either objective alone. Hysteresis and Periodic retain $\lambda = 0.50$ while regulating when candidate changes are applied, allowing their learning and latency outcomes to be compared with the same Greedy reference.

Maximum $J$ follows the same loss and simulated latency trajectory as Greedy $\lambda = 0.75$.The decision records show that Maximum $J$ and Greedy $\lambda = 0.75$ both select $P_2$ in all 95 decisions after the common prefix in all three seeds. For each

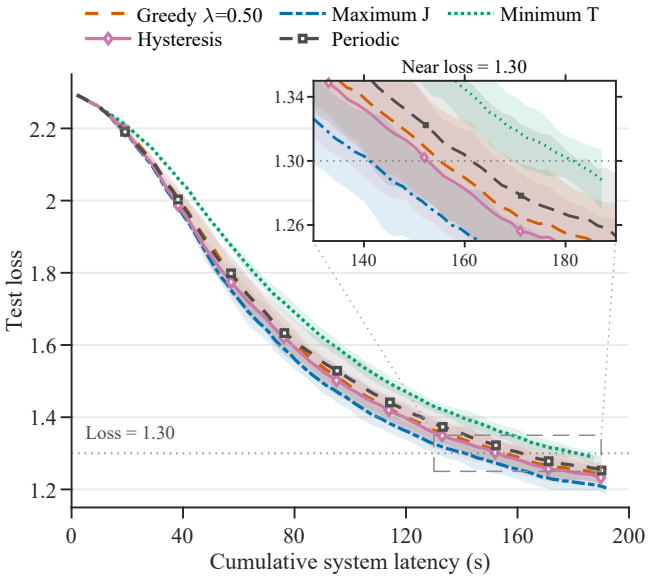



Figure 7: Test loss versus cumulative system latency for the objective and switching controls, with Greedy $\lambda = 0.50$ as the reference. The inset enlarges the region near loss 1.30.

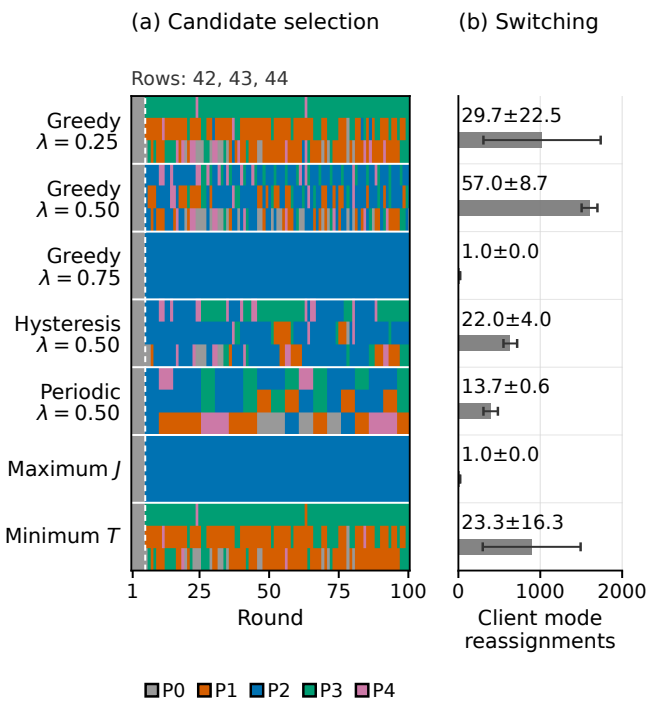



Figure 8: Candidate selection and switching behavior across three seeds. (a) Executed candidate sequences. Within each strategy, rows correspond to seeds 42, 43, and 44 from top to bottom; rounds 1 to 5 share the common $P_0$ initialization. (b) Gray bars and horizontal error bars show the mean and sample standard deviation of total client mode changes. Numbers above the bars report candidate changes between adjacent rounds as mean ± sample standard deviation.

recorded state, their choices satisfy the respective maximum $J$ or maximum utility rule among the feasible members of the five candidate set.

Hysteresis remains close to the Greedy $\lambda = 0.50$ reference, while Periodic shows slower mean loss reduction near the threshold. Hysteresis and Periodic reach loss 1.30 in all three seeds at mean rounds $80.0 \pm 8.9$ and $86.0 \pm 12.1$, respectively. The corresponding switching behavior is examined in Fig. 8. Hysteresis reduces mode changes while maintaining learning performance comparable to Greedy $\lambda = 0.50$, whereas Periodic further reduces switching but exhibits slower loss reduction near the threshold.

### *5.4.3. Mode Selection and Switching*

Figure 8 examines how objective preferences affect candidate choices and how switching controls regulate reconfiguration activity. After the common initialization, Greedy $\lambda = 0.25$ and Minimum $T$ mainly select $P_1$ and $P_3$, although their relative frequencies differ across seeds. Greedy $\lambda = 0.75$ and Maximum $J$ consistently select $P_2$ throughout the remaining 95 rounds in all three seeds. Greedy $\lambda = 0.50$ selects $P_2$ most frequently in each seed while also selecting other candidates.

The switching controls substantially reduce reconfiguration activity relative to Greedy $\lambda = 0.50$. Hysteresis and Periodic reduce the mean number of client mode changes by 60.4% and 75.4%, respectively.Hysteresis and Periodic therefore reduce reconfiguration activity in the direct sense of fewer candidate and client mode changes.

### *5.5. Learning Performance and System Tradeoffs*

Figure 9 compares the final test loss and cumulative recorded system latency of the three Greedy weights with all five fixed candidates after 100 rounds. This comparison examines how different weights affect the final learning–latency tradeoff relative to fixed mode distributions. Table 4 extends the comparison to all 15 strategies and includes communication and computation costs.

To assess the benefits of dynamic selection, each Greedy strategy is compared with a Fixed strategy at a similar final loss level. Greedy $\lambda = 0.25$ and $\lambda = 0.50$ achieve lower mean cumulative latency than Fixed $P_3$ and Fixed $P_4$, respectively. Greedy $\lambda = 0.75$ closely matches Fixed $P_2$ in both final loss and cumulative latency.

All three Greedy mean outcomes lie outside the region dominated by the observed Fixed means. Greedy $\lambda = 0.25$ achieves a lower mean cumulative latency than every Fixed strategy. As $\lambda$ increases from 0.25 to 0.75, the observed mean final loss decreases while mean cumulative latency increases, with $\lambda = 0.75$ approaching Fixed $P_2$. These outcomes associate a stronger preference for historical deviation contribution with lower final loss at a higher latency cost in this experiment.

Pure $FL_{EC}$ achieves the lowest mean final loss, $1.1047 \pm 0.0103$, and the highest mean accuracy over rounds 91–100, $56.65 \pm 0.68\%$, while executing $60{,}000 \pm 0$ local steps and transmitting $17{,}525.8 \pm 0.0$ MB. At the other extreme, pure $FL_E$ has the shortest mean recorded latency, and pure $FL_C$ transmits the least data, but both finish at a mean loss of 1.4146.

Across the means of the 15 observed strategies, local steps and final test loss have Pearson correlation $-0.961$, whereas local steps and mean test accuracy over the last 10 rounds have correlation 0.915. More local work therefore coincides with lower loss and higher accuracy in this matrix. The strong

Table 4: Learning performance and accumulated system costs for all 15 strategies over 100 rounds. Loss is evaluated at round 100, and accuracy is averaged over rounds 91–100 within each seed. Entries are means ± sample standard deviations across three seeds.

| Strategy | Final loss | Accuracy (%) | Cumulative $T$ (s) | Communication (MB) | Local steps | Mode changes |
|---|---|---|---|---|---|---|
| Fixed $P_0$ | 1.2751 ± 0.0268 | 53.24 ± 0.97 | 191.122 ± 2.069 | 10,614.5 ± 0.0 | 32,000 ± 0 | 0 ± 0 |
| Fixed $P_1$ | 1.2896 ± 0.0175 | 54.81 ± 1.66 | 189.498 ± 1.214 | 10,984.8 ± 0.0 | 30,000 ± 0 | 0 ± 0 |
| Fixed $P_2$ | 1.1992 ± 0.0188 | 54.97 ± 0.78 | 191.868 ± 1.628 | 12,836.0 ± 0.0 | 40,000 ± 0 | 0 ± 0 |
| Fixed $P_3$ | 1.2962 ± 0.0320 | 53.47 ± 0.84 | 188.447 ± 5.468 | 9,750.5 ± 0.0 | 30,000 ± 0 | 0 ± 0 |
| Fixed $P_4$ | 1.2413 ± 0.0174 | 54.44 ± 0.17 | 191.253 ± 2.324 | 12,342.4 ± 0.0 | 36,000 ± 0 | 0 ± 0 |
| Greedy $\lambda = 0.25$ | 1.2826 ± 0.0136 | 53.57 ± 0.85 | 187.258 ± 4.344 | 10,506.7 ± 577.6 | 30,693 ± 566 | 1,021.7 ± 715.9 |
| Greedy $\lambda = 0.50$ | 1.2412 ± 0.0323 | 54.29 ± 1.19 | 189.896 ± 1.948 | 11,782.8 ± 275.0 | 35,600 ± 1,393 | 1,601.3 ± 96.6 |
| Greedy $\lambda = 0.75$ | 1.2038 ± 0.0200 | 54.95 ± 0.85 | 191.851 ± 1.668 | 12,724.9 ± 0.0 | 39,600 ± 0 | 20.0 ± 0.0 |
| Hysteresis $\lambda = 0.50$ | 1.2329 ± 0.0313 | 54.64 ± 0.45 | 189.909 ± 3.518 | 11,850.3 ± 522.1 | 36,213 ± 1,633 | 634.0 ± 83.5 |
| Periodic $\lambda = 0.50$ | 1.2526 ± 0.0390 | 53.40 ± 1.67 | 190.154 ± 2.549 | 11,591.5 ± 154.4 | 34,833 ± 1,518 | 394.3 ± 88.2 |
| Maximum $J$ | 1.2038 ± 0.0200 | 54.95 ± 0.85 | 191.851 ± 1.668 | 12,724.9 ± 0.0 | 39,600 ± 0 | 20.0 ± 0.0 |
| Minimum $T$ | 1.2880 ± 0.0192 | 53.28 ± 1.03 | 187.245 ± 4.337 | 10,421.9 ± 510.9 | 30,307 ± 189 | 895.0 ± 597.6 |
| Pure $FL_E$ | 1.4146 ± 0.0224 | 52.31 ± 1.44 | 73.117 ± 0.254 | 10,121.1 ± 0.0 | 20,000 ± 0 | 0 ± 0 |
| Pure $FL_{EC}$ | 1.1047 ± 0.0103 | 56.65 ± 0.68 | 185.558 ± 0.736 | 17,525.8 ± 0.0 | 60,000 ± 0 | 0 ± 0 |
| Pure $FL_C$ | 1.4146 ± 0.0224 | 52.31 ± 1.44 | 83.733 ± 0.358 | 4,937.1 ± 0.0 | 20,000 ± 0 | 0 ± 0 |

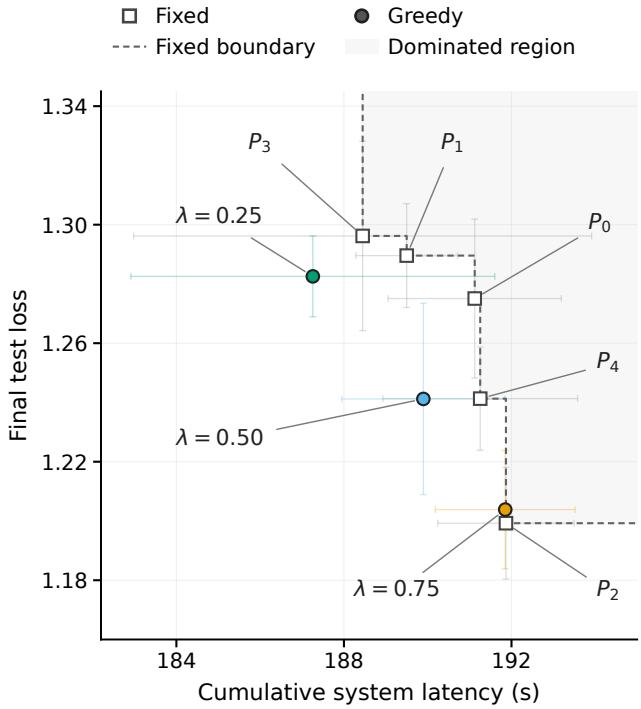



Figure 9: Final test loss versus cumulative system latency after 100 rounds. The dashed staircase marks the observed Fixed frontier, and shading indicates the region dominated by Fixed strategies.

association highlights training workload as an important factor when comparing collaboration strategies. Complementarily, Fig. 5 shows that the initial $J$ ranking of $P_2$, $P_0$, and $P_3$ agrees with their subsequent loss reductions over five rounds.

All 45 strategy runs contain 100 rounds, reach Cloud version 100, record 100 acknowledgements and zero pending units, and remain within $\epsilon = 8$. Across the dynamic strategies, the mean per round Python selection time ranges from 7.34 to 7.68 ms; it is recorded separately and is not included in simulated system latency.

### *5.6. Discussion and Limitations*

The historical aggregate update deviation score summarizes how the weighted historical deviation estimates affect the predicted Cloud update relative to the global reference. The proposed score does not explicitly model the effect of differential privacy noise. Instead, it is constructed from historical updates generated under differential privacy, allowing the effects of privacy protection on update characteristics to be indirectly reflected in the evaluation of candidate mode distributions. This avoids the need to separately estimate the impact of privacy noise on learning performance during candidate evaluation.

Within the finite candidate set, a larger weight on $J$ moves the selector from seed dependent low latency choices toward the larger historical deviation contribution of $P_2$. The resulting high weight strategy attains a competitive loss among the mixed candidates. This supports the limited conclusion that the joint utility produces an observable learning and system cost tradeoff under its declared preferences. It does not establish that larger $J$ causes lower loss, predicts accuracy, or separately identifies the effect of privacy noise.

Several limitations bound this evidence. The reported matrix uses only three seeds, Fashion MNIST, LeNet 5, one topology, and one range of modeled heterogeneity. CIFAR 10, ResNet 18, larger model payloads, and broader topologies require separate validation. The five candidates cover only a small subset of possible client distributions. The network model does not include shared link contention or a physical deployment. Larger models may increase communication, local computation, aggregation, and evaluation of the parameter vectors used by $J$, but this scaling effect is not tested here. Finally, candidate scoring and execution use the same implementation timeline, so the online comparisons are internally consistent.

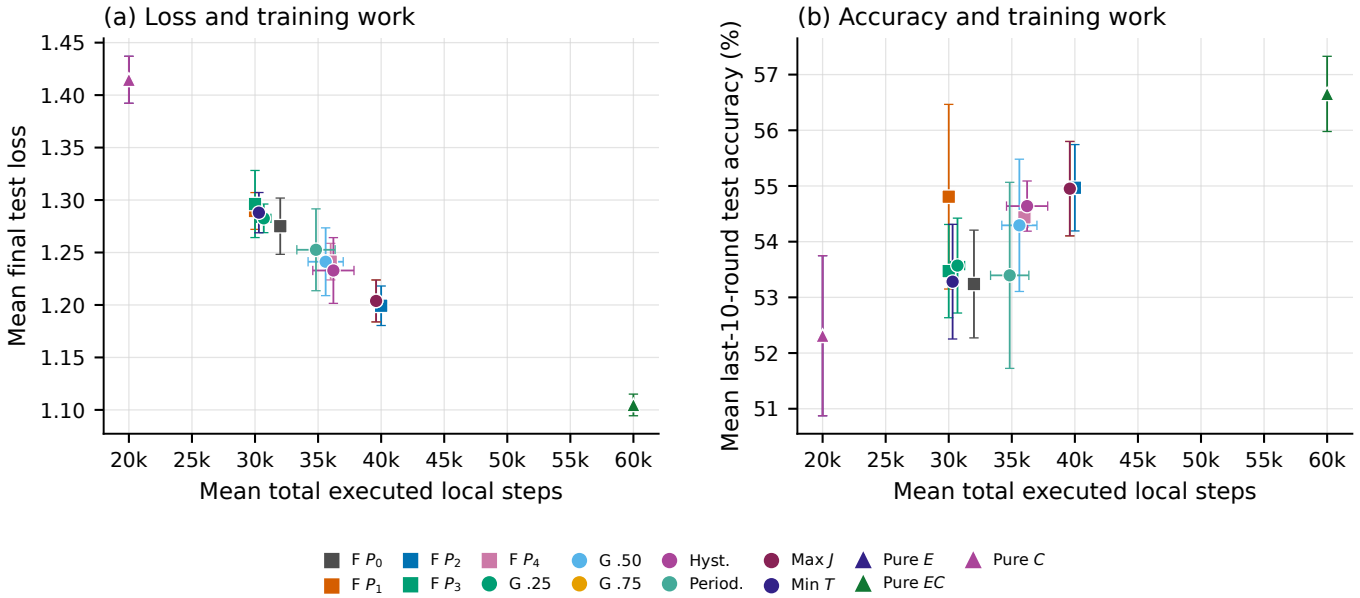



Figure 10: Final learning performance and total executed local steps for the 15 strategies over three seeds. Points and bidirectional error bars show the mean and sample standard deviation. Panel (a) relates local steps to final test loss, and panel (b) relates local steps to final test accuracy.

## 6. Conclusion

This paper presented a dynamic collaboration reconfiguration framework for hierarchical federated learning under resource and lifetime privacy constraints. The framework adapts execution paths and aggregation strategies to the current system state.

Candidate mode distributions are evaluated using historical update deviation contributions and system latency. The two criteria are normalized and combined into a joint utility. At each decision point, Greedy selection chooses the feasible candidate with the highest current joint utility under resource and lifetime privacy constraints. Hysteresis and Periodic selection impose additional switching rules and may retain the current candidate instead of switching to the candidate with the highest current utility. Future work will evaluate the framework using more diverse datasets and model architectures. These evaluations will examine whether historical update deviation contributions remain effective for guiding candidate selection under different learning and system conditions. The computational overhead of candidate selection will also be assessed. Further validation will combine online profiling with physical deployment across the Cloud, Edges, and clients.

## References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial intelligence and statistics, PMLR, 2017, pp. 1273–1282.

[2] Q. Zhao, H. Tang, H. Qiu, W. You, Y. Zhao, J. Lv, Flsp: A federated learning method with self-adaptive privacy for ensuring high model performance in edge computing, Future Generation Computer Systems 183 (2026) 108518. doi:https://doi.org/10.1016/j.future.2026.108518.
URL https://www.sciencedirect.com/science/article/pii/S0167739X26001524

[3] L. Liu, J. Zhang, S. Song, K. B. Letaief, Client-edge-cloud hierarchical federated learning, in: ICC 2020 - 2020 IEEE International Conference on Communications (ICC), 2020, pp. 1–6. doi:10.1109/ICC40277.2020.9148862.

[4] V. A. Le, J. Haga, Y. Tanimura, T. T. Nguyen, Flatec: An efficient federated learning scheme across the thing-edge-cloud environment, Future Gener. Comput. Syst. 175 (2026) 108073. doi:10.1016/J.FUTURE.2025.108073.
URL https://doi.org/10.1016/j.future.2025.108073

[5] Z. Yang, W. Qi, L. Guo, Hierarchical federated learning with device association and resource management for edge networks, Phys. Commun. 77 (2026) 103133. doi:10.1016/J.PHYCOM.2026.103133.
URL https://doi.org/10.1016/j.phycom.2026.103133

[6] T.-M. H. Hsu, H. Qi, M. Brown, Measuring the effects of non-identical data distribution for federated visual classification, arXiv preprint arXiv:1909.06335 (2019).

[7] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, Proceedings of Machine learning and systems 2 (2020) 429–450.

[8] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, A. T. Suresh, Scaffold: Stochastic controlled averaging for federated learning, in: International conference on machine learning, PMLR, 2020, pp. 5132–5143.

[9] A. N. Khan, A. Rizwan, R. Ahmad, Q. W. Khan, X. Li, D. Kim, Accelerating model convergence in

federated learning with layer-wise adaptive weight aggregation, Appl. Soft Comput. 201 (2026) 115676. doi:10.1016/J.ASOC.2026.115676.
URL https://doi.org/10.1016/j.asoc.2026.115676

[10] D. Liao, X. Gao, C. Xu, Mair: Model agnostic instance reweighing for heterogeneous federated learning, IEEE Transactions on Mobile Computing 25 (3) (2026) 4241–4252. doi:10.1109/TMC.2025.3624064.

[11] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, K. Chan, Adaptive federated learning in resource constrained edge computing systems, IEEE Journal on Selected Areas in Communications 37 (6) (2019) 1205–1221. doi:10.1109/JSAC.2019.2904348.

[12] L. Fan, Z. Zeng, X. Zhang, M. Duan, K. Sood, Joint client selection and epoch configuration for heterogeneity-aware federated learning in resource-constrained systems, Future Generation Computer Systems 177 (2026) 108249. doi:https://doi.org/10.1016/j.future.2025.108249.
URL https://www.sciencedirect.com/science/article/pii/S0167739X25005436

[13] C. Ma, X. Li, B. Huang, G. Li, F. Li, Personalized client-edge-cloud hierarchical federated learning in mobile edge computing, Journal of Cloud Computing 13 (12 2024). doi:10.1186/s13677-024-00721-w.

[14] T. Du, X. Chen, G. Kou, Y. Zhao, C. Xu, A clustering federated learning method for cloud-edge personalized model decoupling, Chinese Journal of Computers 48 (2) (2025) 407–432.

[15] C. T. Dinh, N. H. Tran, M. N. H. Nguyen, C. S. Hong, W. Bao, A. Y. Zomaya, V. Gramoli, Federated learning over wireless networks: Convergence analysis and resource allocation, IEEE/ACM Transactions on Networking 29 (1) (2021) 398–409. doi:10.1109/TNET.2020.3035770.

[16] Z. Wu, S. Sun, Y. Wang, M. Liu, B. Gao, Q. Pan, T. He, X. Jiang, Agglomerative federated learning: Empowering larger model training via end-edge-cloud collaboration, IEEE INFOCOM 2024 - IEEE Conference on Computer Communications (2023) 131–140.
URL https://api.semanticscholar.org/CorpusID:266351000

[17] F. Mazloomi, S. Shah-Heydari, K. El-Khatib, A novel multi-server federated learning framework in vehicular edge computing, Future Internet 17 (2025) 315.
URL https://api.semanticscholar.org/CorpusID:280304434

[18] J. Haga, Y. Tanimura, T. T. Nguyen, et al., Sfetec: Split-federated learning scheme optimized for thing-edge-cloud environment, in: 2024 IEEE 20th International Conference on e-Science (e-Science), IEEE, 2024, pp. 1–2.

[19] C. Liu, J. Zhao, Y. Wang, Computation and communication resource optimization for efficient hierarchical federated learning, IEEE Transactions on Networking 34 (2026) 502–517. doi:10.1109/TON.2025.3600163.

[20] B. Yao, C. Li, W. Chen, Y. Chen, Fedcap: Community-aware aggregation and local alignment for personalized federated learning, Future Generation Computer Systems 182 (2026) 108495. doi:https://doi.org/10.1016/j.future.2026.108495.
URL https://www.sciencedirect.com/science/article/pii/S0167739X26001299

[21] M. F. Pervej, R. Jin, H. Dai, Hierarchical federated learning in wireless networks: Pruning tackles bandwidth scarcity and system heterogeneity, IEEE Transactions on Wireless Communications 23 (9) (2024) 11417–11432.

[22] Y. Xu, Y. Liao, H. Xu, Z. Ma, L. Wang, J. Liu, Adaptive control of local updating and model compression for efficient federated learning, IEEE Transactions on Mobile Computing 22 (10) (2023) 5675–5689. doi:10.1109/TMC.2022.3186936.

[23] Y. Su, W. Fan, Q. Meng, P. Chen, Y. Liu, Joint adaptive aggregation and resource allocation for hierarchical federated learning systems based on edge-cloud collaboration, IEEE Transactions on Cloud Computing 13 (1) (2025) 369–382. doi:10.1109/TCC.2025.3530681.

[24] Z. Wei, R. He, X. Chen, C. Song, Dynamic resource allocation for digital twin-enhanced hierarchical federated learning in sustainable internet of things, Computer Communications 248 (2026) 108410. doi:https://doi.org/10.1016/j.comcom.2025.108410.
URL https://www.sciencedirect.com/science/article/pii/S0140366425003676

[25] F. Sattler, K.-R. Müller, W. Samek, Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints, IEEE Transactions on Neural Networks and Learning Systems 32 (8) (2021) 3710–3722. doi:10.1109/TNNLS.2020.3015958.

[26] L. Collins, H. Hassani, A. Mokhtari, S. Shakkottai, Exploiting shared representations for personalized federated learning, in: International conference on machine learning, PMLR, 2021, pp. 2089–2099.

[27] L. Wang, S. Pang, Z. Zhao, X. He, K. Zhang, H. Gui, N. Wang, Fed3scale: A cloud-edge-client tri-scale collaborative semi-supervised hierarchical federated learning framework, Knowledge-Based Systems (2025) 113921.

[28] Q. Ma, Y. Xu, H. Xu, Z. Jiang, L. Huang, H. Huang, Fedsa: A semi-asynchronous federated learning mechanism in heterogeneous edge computing, IEEE Journal on Selected Areas in Communications 39 (12) (2021) 3654–3672. doi:10.1109/JSAC.2021.3118435.

[29] L. Zhu, Z. Liu, S. Han, Deep leakage from gradients, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Vol. 32, Curran Associates, Inc., 2019.
URL https://proceedings.neurips.cc/paper_files/paper/2019/file/60a6c4002cc7b29142def8871531281a-Paper.pdf

[30] C. Dwork, F. McSherry, K. Nissim, A. Smith, Calibrating noise to sensitivity in private data analysis, in: S. Halevi, T. Rabin (Eds.), Theory of Cryptography, Springer Berlin Heidelberg, Berlin, Heidelberg, 2006, pp. 265–284.

[31] J. Chen, H. Jiang, Q. Hu, Utility-enhanced personalized privacy preservation in hierarchical federated learning, IEEE Transactions on Mobile Computing 24 (6) (2025) 5264–5279. doi:10.1109/TMC.2025.3531919.

[32] X. Xu, Y. Li, L. Lu, T. Wang, Adpfl: Adaptive differential privacy-enhanced federated learning, IEEE Internet of Things Journal 12 (23) (2025) 50682–50692. doi:10.1109/JIOT.2025.3611410.

[33] H. Wang, J. Mai, L. Zhang, L. Cui, F. R. Yu, J. Liu, Toward double-layer data privacy in communication-efficient hierarchical federated learning: A client sampling approach, IEEE Journal on Selected Areas in Communications 44 (2026) 2158–2172. doi:10.1109/JSAC.2025.3636844.

[34] A. S. Kalafatelis, V. Nikolakakis, N. Tsoulakos, P. Trakadas, Privacy-preserving hierarchical federated learning over data spaces, in: 2025 IEEE International Conference on Big Data (BigData), 2025, pp. 6426–6435. doi:10.1109/BigData66926.2025.11400909.

[35] M. M. Islam, W. M. Abdullah, B. N. Saha, Privacy-preserving hierarchical fog federated learning (pp-hffl) for iot intrusion detection, Sensors (Basel, Switzerland) 25 (2025).
URL https://api.semanticscholar.org/CorpusID:283618612

[36] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS '16, Association for Computing Machinery, New York, NY, USA, 2016, p. 308–318. doi:10.1145/2976749.2978318.
URL https://doi.org/10.1145/2976749.2978318

[37] I. Mironov, Rényi differential privacy, in: 2017 IEEE 30th Computer Security Foundations Symposium (CSF), 2017, pp. 263–275. doi:10.1109/CSF.2017.11.

[38] C. Dwork, A. Roth, The algorithmic foundations of differential privacy, Foundations and Trends in Theoretical Computer Science 9 (3–4) (2014) 211–407. doi:10.1561/0400000042.

[39] T. van Erven, P. Harremos, Rényi divergence and kullback-leibler divergence, IEEE Transactions on Information Theory 60 (7) (2014) 3797–3820. doi:10.1109/TIT.2014.2320500.